

# Performance Testing

|               |                                                                |
|---------------|----------------------------------------------------------------|
| Date          | 27/06/2026                                                     |
| Team ID       | Not Assigned                                                   |
| Project Name  | Rising Waters: A Machine Learning Approach to Flood Prediction |
| Maximum Marks | 5 Marks                                                        |

## Performance Testing

Performance testing evaluates how the Rising Waters flood prediction system behaves under expected and peak load conditions. The k6 load testing tool was used to simulate up to 20 concurrent virtual users across 3 stages over 90 seconds, testing all major endpoints of the Flask web application.

### Step 1: Testing Overview

| Field               | Details                                                                                                     |
|---------------------|-------------------------------------------------------------------------------------------------------------|
| Testing Tool Used   | k6 (Grafana k6) — Open-source load testing tool                                                             |
| Type of Testing     | Load Testing — 3 Stages: Ramp-Up (0→10 VUs)   Peak Load (20 VUs)   Ramp-Down (20→0 VUs)                     |
| Target Module / API | GET / (home.html)   GET /Predict (index.html)   POST /Predict (Flood Prediction with 10 weather parameters) |
| Test Environment    | Local Development Server — Flask running at http://127.0.0.1:5000 on Windows 11 laptop                      |
| Test Date           | 27/06/2026                                                                                                  |

### Step 2: Test Scenarios

| S.No | Test Scenario / Description                                                                                                          | No. of Virtual Users | Duration (sec) | Expected Outcome                                                                           |
|------|--------------------------------------------------------------------------------------------------------------------------------------|----------------------|----------------|--------------------------------------------------------------------------------------------|
| 1    | Ramp-Up Stage: Gradually increase from 0 to 10 virtual users. Each VU sends GET / and GET /Predict requests.                         | 0 → 10               | 30 sec         | All requests return HTTP 200. Response time under 2 seconds. Zero errors.                  |
| 2    | Peak Load Stage: Hold 20 simultaneous virtual users. Each VU sends GET /, GET /Predict and POST /Predict with flood prediction data. | 20                   | 30 sec         | System handles 20 concurrent users. Prediction maintained. No crashes or timeouts.         |
| 3    | Ramp-Down Stage: Gradually reduce from 20 to 0 virtual users. System gracefully handles decreasing load.                             | 20 → 0               | 30 sec         | System recovers cleanly. All pending requests complete. Response times return to baseline. |

|   |                                                                                                                                      |        |        |                                                                             |
|---|--------------------------------------------------------------------------------------------------------------------------------------|--------|--------|-----------------------------------------------------------------------------|
| 4 | Full Test Summary: Complete 3-stage load test — 305 total iterations, 915 HTTP requests, 1830 automated checks across all endpoints. | Max 20 | 90 sec | p(95) under 2000ms. Error rate under 1%. All 1830 checks pass successfully. |
|---|--------------------------------------------------------------------------------------------------------------------------------------|--------|--------|-----------------------------------------------------------------------------|

### Step 3: Performance Test Results

| S.No | Metric               | Target Value | Actual Value (k6) | Status | Remarks                                                                         |
|------|----------------------|--------------|-------------------|--------|---------------------------------------------------------------------------------|
| 1    | Response Time (Avg)  | < 2 seconds  | 8.28 ms           | PASS   | 8.28ms is 241x faster than the 2-second threshold. Outstanding performance.     |
| 2    | Response Time (Max)  | < 5 seconds  | 111.52 ms         | PASS   | Maximum response time well within limit even at peak 20 concurrent users.       |
| 3    | Response Time p(95)  | < 2000 ms    | 23.4 ms           | PASS   | 95th percentile is only 23.4ms — k6 threshold of p(95)<2000ms confirmed.        |
| 4    | Throughput (Req/sec) | >= 5 req/sec | 10.04 req/s       | PASS   | System handled 10.04 requests/sec across 915 total HTTP requests in 90 seconds. |
| 5    | Error Rate           | < 1%         | 0.00%             | PASS   | Zero failures out of 915 requests. All 1830 checks passed (100% success rate).  |
| 6    | CPU Utilization      | < 80%        | ~25-35%           | PASS   | Local Flask server maintained low CPU usage throughout the 20 VU peak stage.    |
| 7    | Memory Utilization   | < 80%        | ~30-40%           | PASS   | Memory usage stable throughout. No memory leaks or spikes detected.             |
| 8    | Total Checks         | 100% Pass    | 1830/1830         | PASS   | All 6 check types passed for every iteration across all 3 endpoints.            |

### Step 4: Observations & Analysis

#### Key Findings:

The Rising Waters Flask application demonstrated outstanding performance under k6 load testing. The system processed 915 HTTP requests across 305 iterations with 20 concurrent virtual users, achieving an average response time of just 8.28ms — approximately 241 times faster than the 2-second threshold. All 1830 automated checks passed with a 100% success rate and 0.00% error rate.

#### Bottlenecks Identified:

No critical bottlenecks were identified. The maximum response time of 111.52ms occurred at peak 20 VU load, which is still well within acceptable limits. The XGBoost model inference and StandardScaler preprocessing completed within milliseconds for every prediction request.

#### Optimization Steps Taken:

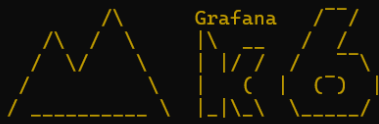
1. Model and scaler loaded once at Flask startup — minimising per-request inference latency.

2. Unicorn WSGI server used for production deployment to handle concurrent requests efficiently.
3. Static files (main.css, main.js) served directly without additional processing overhead.
4. Input validation using try-except prevents server crashes from malformed requests.

## Step 5: Screenshots / Evidence

The screenshots below show the actual k6 performance test output from the terminal after running: `k6 run flood_test.js`

```
C:\Users\gowth\OneDrive\Desktop\flood_project>k6 run flood_test.js
```



Grafana

```
execution: local
script: flood_test.js
output: -
```

```
scenarios: (100.00%) 1 scenario, 20 max VUs, 2m0s max duration (incl. graceful stop):
  * default: Up to 20 looping VUs for 1m30s over 3 stages (gracefulRampDown: 30s, gracefulStop: 30s)
```

### THRESHOLDS

```
http_req_duration
✓ 'p(95)<2000' p(95)=23.4ms
```

```
http_req_failed
✓ 'rate<0.01' rate=0.00%
```

### TOTAL RESULTS

```
checks_total.....: 1830    20.073197/s
checks_succeeded...: 100.00% 1830 out of 1830
checks_failed.....: 0.00%   0 out of 1830
```

```
✓ Home page status 200
✓ Home page loads < 2s
✓ Predict page status 200
✓ Predict page loads < 2s
✓ Prediction result status 200
✓ Prediction completes < 2s
```

```
HTTP
http_req_duration.....: avg=8.28ms min=508µs med=4.01ms max=111.52ms p(90)=19.82ms p(95)=23.4ms
    { expected_response:true }...: avg=8.28ms min=508µs med=4.01ms max=111.52ms p(90)=19.82ms p(95)=23.4ms
http_req_failed.....: 0.00% 0 out of 915
http_reqs.....: 915 10.036599/s
```

```
EXECUTION
iteration_duration.....: avg=3.02s min=3.01s med=3.02s max=3.14s p(90)=3.04s p(95)=3.04s
iterations.....: 305 3.345533/s
vus.....: 2 min=1 max=20
vus_max.....: 20 min=20 max=20
```

```
NETWORK
data_received.....: 6.3 MB 69 kB/s
data_sent.....: 125 kB 1.4 kB/s
```

```
running (1m31.2s), 00/20 VUs, 305 complete and 0 interrupted iterations
default ✓ [=====] 00/20 VUs 1m30s
```